

Server/PC Remote Login/Use Access

Table of Contents

Objective (What did we do?):	3
Steps (How did we do it?)	4
Outcome (Why did we do it?)	8

Objective

(What did we do?):

Setting up the servers to have access to them from anywhere through the cockpit administrative interface.

From here, we will leave out computers tucked in somewhere as the work continuing forward will be done remotely.

We are downloading MariaDB so that we have access to an open-source community developed database management system

Steps

(How did we do it?)

1. Update OS

```
dnf upgrade --refresh
```

Note: A Kernel is just the part that communicates the hardware to the software

2. Reboot

```
shutdown -r now
```

3. Check?*

```
systemctl __ httpd
```

4. Make a copied directory of the system

```
mkdir origconfigs
```

5. Go to the directory to copy files

```
cd /etc/ssh  
cp ssh_config /root/origconfigs
```

6. Go to text editor

```
emacs sshd_config
```

All/most of your config files are in /etc/

7. Go to '#Port 22' and change it to 'Port 44444' and save with (ctrl x + ctrl c) followed by y

8. Restart sshd

```
systemctl restart sshd
```

9. Check that it restarted

```
systemctl status sshd
```

10. Enable the firewall to receive packets through the wall through Port 44444

```
Firewall-cmd --zone=public --add-port=44444/tcp --permanent
```

11. Reload

```
firewall-cmd --reload
```

12. Look at the list of everything

```
firewall-cmd --list-all
```

Now you can see this from another computer and log in using the log in credentials and Port number.

13. On another computer, in a terminal, you can type ssh -l user_name IP_Address -p port_#

```
Ssh -l Copernicus 142.i forgot rest -p 44444
```

You can not log in with the root from another computer.

14. You can become root once logged into your user account Copernicus, to become root, type:

```
su -
```

and then the password. You are now logged into root.

15. We are installing cockpit. It is a default web administrator interface log in for a RedHat box.

```
dnf install cockpit
```

16. Check system status on cockpit

```
systemctl status cockpit
```

17. Start cockpit

```
systemctl start cockpit
```

18. Type on on webpage on another computer

```
IP_Address :9090
```

```
142.3.24.32:9090
```

Hazzard page output. It will say that your connection is not private. You can not view cockpit with anything other than https. Click at the bottom to proceed

19. You now have an interface to log into your Rock Box. Log into it with your username

20. When logged in you will see in the left-hand menu at the bottom terminal. We want this. From here you can log into the server as root. We now can log into the server through a keyboard at school and the webpage at home.

21. Unplug the keyboard and stuff and leave it plugged in on the floor.

22. Install the ultimate bundle, MySQL

```
dnf install mariadb-server mariadb
```

23. Start MariaDB

```
Systemctl start mariodb
```

24. Access MariaDB port

```
telnet 127.0.0.1 3307
```

You should be connected. We will not open the firewall in the port for safety reasons. Typically, on a lamp box, we do not open 3303, except for only the machine that is connecting to it, the server. That is why it has the loop back address.

25. Enable it to start every time the machine starts

```
Systemctl enable mariadb
```

It will have created some **symlinks**

You could go to the location of the folder and see a list of the stuff in the folder.

There is a directory called **'multi-user.target.wants'**, if you go to it and do **'ls'**, this is where the **symlinks** are. Enable is what starts all of these up when started

26. Go back to root folder

```
cd ~
```

27. If you do **'ls'**, you can see **'origconfigs'**

28. To secure mariadb install this

```
mariadb-secure-installation
```

We do not have a password since we have not created a root on it.. since it is not the root on your box.

Click **enter**, followed by **n** for switching to **unix_socket** authentication

29. Change root password, click yes and make one

30. Remove anonymous users, yes

31. Disallow root login remotely, yes

32. Remove test database and access to it, yes

33. Reload privilege tables now, yes

34. You are complete, you can now use the database. In MySQL you need semicolons to finish off or get an arrow waiting for more.

```
mariadb -u root;
```

35. Set password

```
mariadb -u root -p;
```

36. Select version();

37. Check databases

```
show databases;
```

38. Quit. It will take you back to the commend prompt prior to entering.

```
quit;
```

39. So we have Linus, Apache, MySQL, and now been php

```
Dnf install php php-cli php-pdo php-fpm php-json php-mysqlnd
```

NOW, we have the full gangster stack that changed the internet.

40. Default directory for apache

```
Cd /var/www/html
```

41. Create in text editor a php page

```
emacs info.php
<?php top to start
    Phpinfo();
?>
```

42. We need to restart httpd

```
Systemctl restart httpd
```

43. Open it

```
http://142.3.24.32/info.php
```

44. Make database, first login

```
Mariadb -u root -p  
Mariadb database mydatabase;
```

45. Use mydatabase

46. Create data in it, a table

```
Create table Employees (IDNum int, LastName varchar(255), FirstName  
                        varchar(255), Position varchar(255));
```

47. Show tables;

48. Show columns from Employees;

49. Insert into Employees (IDNum,...) Values ('1', ...)

50. Select * from Employees;

Will show all rows in Employees table

51. Update the table

```
Update Employees set Position='CEO' where IDNum=1;
```

Outcome

(Why did we do it?)

The server was transitioned into a remote, secure web server that can be accessed through the internet from home or wherever in Canada that the University has permitted.

The default SSH port was changed to 44444 which is one that we can be sure that will not be used.

The firewall was updated, which secured the remote entry from basic attacks.

Cockpit was also installed which provided us a user interface dashboard, allowing us to access the root terminal from anywhere.

MariaDB and PHP was downloaded which deployed our LAMP stack (Linux, Apache, MariaDB, PHP).

The database management system was secured and were provided basic entry into SQL commands to create and manipulate tables which was provided for a programming assignment.

In summary, the infrastructure has been built for dynamic hosting for database web applications.